

# Team Project

California State University, Fresno

Department of Computer Science

**CSCI 115 - Algorithms and Data Structures**

Spring 2024

Sarah Garner, Guillermo Prado, Cory Heins, and Anthony Sayre

# Part 1

## I. INTRODUCTION

In this report, we will analyze the following common sorting algorithms using both theoretical knowledge and experimental results: Insertion Sort, Selection Sort, BubbleSort, QuickSort, MergeSort, HeapSort, Counting Sort, and Radix Sort. We will compare the experimental results to each other and then compare and contrast the running times of algorithms obtained experimentally with the expected time complexities from theory. Lastly, we will determine whether or not the number of computations is correlated with execution time.

## II. BACKGROUND: THEORETICAL TIME COMPLEXITIES

### A. Insertion Sort

Insertion sort iterates through an array (A) of size  $n$  starting from the second element. Elements larger than the selected element are moved one position to the right until all elements to the left of the selected element are smaller and all elements to the right are larger. (Algorithm 1)

| Code                                                    | Cost  | Times                  |
|---------------------------------------------------------|-------|------------------------|
| for $j \leftarrow 2$ to $n$                             | $c_1$ | $n$                    |
| do $key \leftarrow A[j]$                                | $c_2$ | $n-1$                  |
| insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$ | 0     | $n-1$                  |
| $i \leftarrow j - 1$                                    | $c_4$ | $n-1$                  |
| while $i > 0$ and $A[i] > key$                          | $c_5$ | $\sum_{j=2}^n t_j$     |
| do $A[i+1] \leftarrow A[i]$                             | $c_6$ | $\sum_{j=2}^n t_j - 1$ |
| $i \leftarrow i - 1$                                    | $c_7$ | $\sum_{j=2}^n t_j - 1$ |
| $A[i+1] \leftarrow key$                                 | $c_8$ | $n-1$                  |

#### Algorithm 1: Insertion Sort [1]

]Pseudocode of the computational steps needed for Insertion Sort.

1) *Worst Case*: In this case, the list to be sorted is in descending order. Therefore,  $A[i]$  (Algorithm 1) is always greater than the key meaning the while loop runs  $j$  times for each iteration of the for loop. Therefore, the worst case scenario has a time complexity of  $\Theta(n^2)$ . [1]

2) *Best Case*: In the best case scenario, the list has already been sorted in ascending order. Therefore,  $A[i] \leq key$  (Algorithm 1). Thus, none the two lines inside of the while loop run. Additionally, the conditions of the while loop are only checked once with each iteration of the for loop. Thus, the conditions of the while loop are only checked  $n - 1$  times. (Algorithm 1) This results on a runtime of  $\Theta(n)$  rather than  $\Theta(n^2)$ . [1]

3) *Average Case*: The average time complexity is  $O(n^2)$  as on average, Insertion Sort requires approximately  $(n^2)/4$  comparisons and swaps to sort an array of size 'n'. [1]

### B. Bubblesort

Bubblesort sorts elements in ascending order by iterating through a list of elements, swapping adjacent elements if the element on the left is greater than the element on the right. (Algorithm 2) This swapping guarantees the element to the far right is always the largest element each iteration. The elements passed to the right of the array then form an already sorted list and the remaining elements are iterated through until they are sorted.

| Code                                        | Cost  | Times                  |
|---------------------------------------------|-------|------------------------|
| for $i \leftarrow 1$ to $n$                 | $c_1$ | $n+1$                  |
| do for $j \leftarrow n$ downto $i+1$        | $c_2$ | $\sum_{i=1}^n (n-i+1)$ |
| do if $A[j] < A[j-1]$                       | $c_3$ | $\sum_{i=1}^n (n-i)$   |
| then exchange $A[j] \leftrightarrow A[j-1]$ | $c_4$ | $\sum_{i=1}^n (n-i)$   |

### Algorithm 2: BubbleSort [1]

]Pseudocode of the computational steps needed for Bubble Sort.

1) *Best, Worst, and Average Cases:* The runtime is  $\Theta(n^2)$  in all cases because the doubly-nested for loop runs a set number of times regardless of the numbers in the input array. (Algorithm 2) The time complexity primarily is affected by the length of the list.

### C. Selection Sort

Selection sort repetitively finds the minimum element from the unsorted portion of an array and moves it to the beginning. This creates a sorted sub-array on the left and an unsorted sub-array on the right. This process is then applied to the unsorted list until all elements are sorted.

1) *Worst, Best, and Average Cases:* Similar to insertion sort, the runtime is  $\Theta(n^2)$  in all cases. [1] The array is iterated through a set number of times regardless of the elements in the input array.

### D. Quicksort

Quicksort is a sorting algorithm using a divide-and-conquer strategy. An element is selected to partition the array into two sub-arrays. Elements smaller than the partition are moved to the left and elements larger than the right. Then, the sub-arrays are recursively partitioned and sorted using the above methods until all elements are sorted. The time complexity of quicksort depends upon the partitioning function. [1]

1) *Worst Case:* In the worst case, the array is continuously split into two subarrays, one with only one element and the other with all of the other elements. This occurs when the pivot is the greatest or smallest element in the subarray. For the given partitioning function, this would occur continuously if the array elements were in descending or ascending order. The resulting time complexity of the worst case would be  $\Theta(n^2)$ . [1]

2) *Best Case:* In the best case, the array is split approximately in half with each recursive call to Quicksort. This would result in a time complexity of  $\Theta(n \lg n)$  by the Master Theorem as there are two recursive calls to Quicksort, each taking half of the array. [1] Thus, this best case scenario occurs when the pivot is selected such that the array is divided equally. A strategy to achieve this is to pick the median of the array or a random element as the pivot. [1]

3) *Average Case:* The average time complexity is  $O(n \lg(n))$ . [1] This is found by the expected number of comparisons, which uses the probability of comparing any two elements in an array. [1]

### E. Merge-Sort

Merge sort is a sorting algorithm that uses a divide-and-conquer strategy to organize elements. It works by recursively dividing the unsorted list until there are  $n - 1$  sub-arrays each with one element. Next, these sub-arrays are repetitively merged into larger sub-arrays until there is one sorted array. [1]

1) *Worst, Best, and Average Cases:* Merge sort has a consistent time complexity of  $\Theta(n \lg n)$  regardless of the order of elements in the input array. [1] This is because the array is split in half recursively, which results in  $\log n + 1$  levels and for each level there is a merging time of  $\Theta(n)$ . Thus, the runtime of merge sort primarily depends upon the size of the array it is sorting. [1]

## F. Heapsort

| Pseudocode                                   | Line |
|----------------------------------------------|------|
| Alg.: Heapsort(A)                            | 1    |
| Build-Max-Heap(A)                            | 2    |
| for $i \leftarrow \text{length}[A]$ downto 2 | 3    |
| do exchange $A[1] \leftrightarrow A[i]$      | 4    |
| Max-Heapify(A, 1, i-1)                       | 5    |

Algorithm 3: Heapsort [1]  
]Pseudocode of Heap Sort.

Heap sort (Algorithm 3) uses a binary max heap data structure to sort elements. First, it builds a max heap structure using a Build-Max-Heap function. This ensures that the largest data element is the root element. The algorithm then repeatedly removes the root of the heap, places it at the end of the array, and uses the Max-Heapify function to restore the heap property, with the next largest element as the root of the heap. This process continues until all elements are sorted.

1) *Worst, Best, and Average Cases:* The time complexity of Heapsort (Algorithm 3) is  $\Theta(n \log n)$  in the best, worst, and average cases. [1] This is because Heapsort calls Build-Max-Heap which has a time complexity of  $O(n \log n)$ . [1] The Build-Max-Heap function uses a for loop, which has a time complexity of  $O(n)$ , to call Max-Heapify, which has a time complexity of  $O(\log n)$ , on all parent nodes. [1] Max-Heapify has a runtime of  $O(\lg n)$  as the height of the heap is  $\lg n$  and Max-Heapify swaps a node with its largest child from the root down to a leaf. [1]

## G. Counting Sort

Counting sort [1] is a non-comparison-based sorting algorithm. It uses an array of element frequencies, and array of indices, and an output array, to sort elements. It begins by counting the number of elements that have distinct key values and then storing each count in an array with each index in the array corresponding to a specific element. This creates the frequency array. The algorithm then calculates the histogram of key frequencies by summing up values in the frequency array from left to right. This creates an index array, with each element in the index array corresponding to the final indices of elements in the output array. Lastly, the algorithm iterates through the frequency and index array, decrementing both by one each time, and placing elements in the output array using their corresponding index in the index array.

The time complexity of Counting Sort is  $O(n + k)$ , where  $n$  is the number of elements in the input array and  $k$  is the range of the input. [1] This is because the algorithm iterates through arrays of size  $n$  (for the input and output arrays) and  $k$  (for the frequency and index arrays). [1]

1) *Worst Case:* In the worst case scenario,  $k$  is a lot larger than  $n$ . This could result in a greater runtime than  $O(n)$  as the algorithm must iterate through an array of length  $k$  multiple times in order to tally up the number of each distinct element and calculate each element's indices in the ordered array.

2) *Best Case:* The best case scenario is when  $k \leq O(n)$  as this would ensure a time complexity of  $O(n)$ . [1]

3) *Average Case:* The average time complexity is  $O(n)$  as  $k$  would have to be significantly larger than  $n$  (i.e.  $k = n^2$ ) in order to have a different runtime.

## H. Radix Sort

Radix sort is a non-comparison-based sorting algorithm similar to counting sort. Unlike counting sort, using the largest element in the data, separate arrays are created for each digit (i.e. tens place, hundreds

place, etc.). The elements are then sorted by calling counting sort on the digits from left to right.

The time complexity for Radix Sort is  $O(d * (n + k))$ , where  $n$  is the number of elements to be sorted,  $k$  is the number of possible values for each digit, and  $d$  is the number of digits in the maximum number in the input array. [1] This is because the arrays are of size  $n$  and  $k$  but the number of arrays is dependent on  $d$ . [1]

1) *Best Case*: The best case is when  $k$  and  $d$  are small enough such that  $k = O(n)$  and  $d = O(1)$ . [1] This results in a total time complexity of  $O(n)$ .

2) *Worst Case*: The worst case is when  $k > cn$  or  $d \geq n$ . [1] This results in a total time complexity greater than  $O(n)$ .

3) *Average Case*: The average case has a time complexity of  $O(n)$  as  $k$  would have to be a lot greater than  $n$  or  $d \geq n$  in order to have a time complexity other than  $O(n)$ .

### III. METHODOLOGY

#### A. Data Description

Our experimental setup involves three distinct data sets, each tailored to test the sorting algorithms under different scenarios—Sorted, Random, and Reversed. These datasets were chosen to help us understand how each sorting algorithm performs under ideal, average, and challenging conditions. The input sizes for these sets are 500, 50,000, and 500,000, allowing us to evaluate the scalability of the algorithms from relatively small to large inputs. This range is designed to provide a clear picture of both the lower and upper bounds of the algorithms' runtime performances.

#### B. Data Generation

We created a program that generates our datasets. The program prompts the user to enter the size of the vector and the filename for the output file. Next, the program generates three types of vectors: sorted, random, and reversed sorted. For each type, it populates a vector accordingly: sorted from 1 to  $n$ , random order using `std::shuffle`, and reversed sorted from  $n$  to 1. The program writes the generated vectors to a file. It utilizes `ofstream` to create and open a file and writes each vector type along with its title to the file. The program performs error handling for file opening. If the file cannot be opened, it displays an error message and exits the program. After generating a file, the program displays a message with a successful generation and name of file.

The dataset sizes (500, 50,000, 500,000) were chosen to cover small, medium, and large datasets, aligning with the time complexity of sorting algorithms. Smaller datasets allow evaluation of quadratic-time algorithms like insertion and selection sort, while larger datasets provide insights into the efficiency of algorithms with better time complexities, such as merge and quicksort.

#### C. Set-Up

The experiments were conducted on a 14-inch MacBook Pro equipped with an M3 Max chip, featuring a 14-core CPU, 30-core GPU, 16-core Neural Engine, and 36GB of unified RAM. This powerful setup ensures that our timings are accurate and reliable. We chose a Unix environment for its efficiency and direct hardware interface, avoiding the potential overhead of virtualization. However, we noted that cache effects could influence the timing, especially since the same data sets were repeatedly used across different algorithms.

For the timing mechanism, we utilized the chrono library in our code, which captures high-resolution time measurements in microseconds. The implementation involved a function that takes the duration of each algorithm run, computes the average of the three runs, and stores it in a Long Long data type. This setup not only ensures precision but also allows for consistent application across various algorithms without altering the datasets between runs.

Each sorting algorithm was tested using the same three data sets, ensuring a fair and consistent comparison. The sorted Data Set is the best case scenario for most sorting algorithms. The random Data Set represents the average case scenario. Lastly, the reversed Data Set is typically the worst case scenario as the algorithms must reverse the order of the dataset elements.

#### *D. Data Analysis*

We used the time it took for our algorithms to sort arrays of various sizes and compositions in order to compare the efficiency of different sorting algorithms to each other. Additionally, we compared the results of our experiment with the expected time complexities through three methods. Firstly, we plotted the number of steps each algorithm took versus the input array size and then the time each algorithm took to run versus the input array size. This allowed us to see whether algorithms with similar theoretical time complexities had similar runtimes in the results of our experiment. Secondly, using bar charts, we compared the theoretical number of steps with the number of clock cycles each algorithm took, as estimated using our experimental times and our computer's CPU speed. Next, we tested whether the time complexity of our algorithms in our experiment matched their theoretical time complexities by plotting the experimental time divided by the theoretical time complexity on the y-axis, and the size of the input array on the x-axis to see if there was a straight line, indicating a relationship.

### IV. RESULTS AND DISCUSSION

#### *A. Best Sorting Algorithm*

When comparing the experimental and theoretical graphs for the best-case run time of sorting algorithms on data arrays, it's apparent that theoretical run times exhibit a more dramatic divergence between the algorithms as the input size increases. However, in the experimental data, the run times are much closer together. This suggests that in a practical setting, many sorting algorithms have similar efficiency for sorted data, which could be due to optimized implementation details that theoretical models do not account for. The theoretical graph underscores the worst-case complexity, while the experimental graph reflects more realistic, optimized conditions. Furthermore, experimental graphs show differences in the speed algorithms with the same theoretical time complexity, pointing out the importance of details in implementation.

Based upon our theoretical background, we expected either Insertion Sort, Counting Sort, and Radix Sort to perform the best when sorting the already sorted arrays as the algorithms have a time complexity of  $\Theta(n)$  in that scenario. In the results from our experiment, Insertion Sort was always the fastest sorting algorithm for pre-sorted arrays, irrespective of array size. (Tables I-III)(Figures 1-3). Insertion Sort may have performed better than Counting Sort and Radix Sort in practice due to the in-place nature of the sort, meaning that time wasn't spent initializing arrays. Counting Sort was the fastest algorithm for sorting both reversed arrays and random arrays of all sizes. (Tables I-III)(Figures 1-3) Counting Sort may have been faster than Radix Sort as elements in our dataset were such that sorting elements by digit didn't make Radix Sort more efficient than counting sort, and may have expended time initializing more arrays than required with Counting Sort. Heap Sort and Bubble Sort did unexpectedly worse when sorting larger arrays of elements. (Tables I-III)(Figures 1-3)

|           | Sorted | Reversed | Random |
|-----------|--------|----------|--------|
| Insertion | 0      | 123      | 68     |
| Selection | 378    | 448      | 493    |
| Bubble    | 122    | 368      | 286    |
| Merge     | 53     | 42       | 66     |
| Quick     | 20     | 17       | 29     |
| Heap      | 32     | 24       | 35     |
| Counting  | 4      | 3        | 4      |
| Radix     | 9      | 6        | 8      |

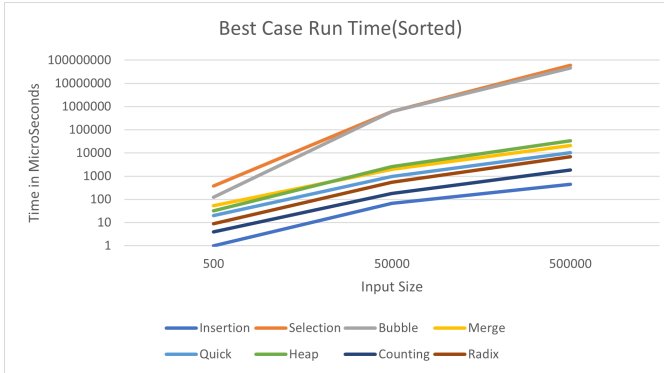
TABLE I: Time (Microseconds) To Sort Array (n=500)

|           | Sorted | Reversed | Random  |
|-----------|--------|----------|---------|
| Insertion | 67     | 570000   | 258590  |
| Selection | 609410 | 1072843  | 771497  |
| Bubble    | 621360 | 1876695  | 1956613 |
| Merge     | 1954   | 2008     | 4033    |
| Quick     | 958    | 1048     | 2120    |
| Heap      | 2616   | 2629     | 2876    |
| Counting  | 179    | 241      | 239     |
| Radix     | 537    | 526      | 545     |

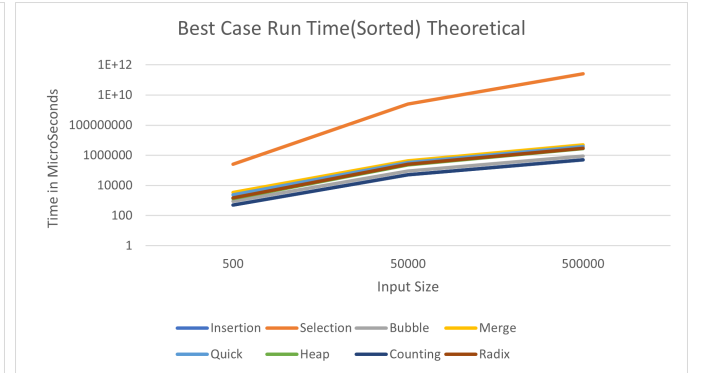
TABLE II: Time (Microseconds) To Sort Array (n=50,000)

|           | Sorted   | Reversed  | Random    |
|-----------|----------|-----------|-----------|
| Insertion | 441      | 50730088  | 25489301  |
| Selection | 59710057 | 64561231  | 71543661  |
| Bubble    | 46676860 | 317886171 | 245282190 |
| Merge     | 20743    | 21836     | 46931     |
| Quick     | 10191    | 11290     | 25189     |
| Heap      | 33728    | 32990     | 41826     |
| Counting  | 1847     | 2126      | 2661      |
| Radix     | 6850     | 6835      | 7142      |

TABLE III: Time (Microseconds) To Sort Array (n=500,000)



(a) Experimental Results



(b) Theoretical Results

Fig. 1: Comparison of Results: Sorted Arrays

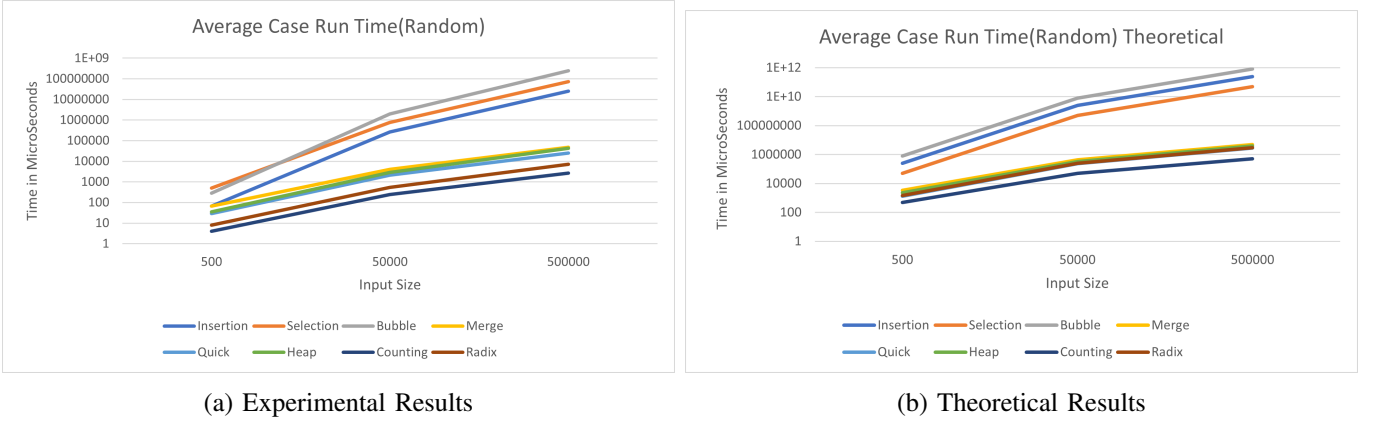


Fig. 2: Comparison of Results: Random Arrays



Fig. 3: Comparison of Results: Reversed Arrays

### B. Comparison of Theoretical and Experimental Results

For a preliminary comparison of our experimental results with the theoretical time complexities, we compared the clock cycles from our experiment with the theoretical time complexity. Figures 4 through 11 demonstrate how the number of steps calculated by the theoretical time complexity often acts as an upper bound for the actual number of steps taken by an algorithm. Therefore, any times calculated using the time complexity would be greater than the actual runtimes.

To test whether or not our algorithm's experimental times conformed with the asymptotic analysis, we divided the experimental times by the time complexities in each case. (Figures 12 - 19) Our results were mixed. The following algorithms clearly followed the predicted time complexity: Selection Sort (Figure 14) for random and sorted arrays, MergeSort (Figure 15), QuickSort (Figure 16), HeapSort (Figure 17) for reversed and sorted arrays, and Counting Sort (Figure 18) for sorted arrays. Even when we didn't obtain a constant value, most of the graphs had very small values for the experimental time divided by the time complexity, indicating that the time complexity wasn't too far off. Discrepancies between our results and the theoretical time complexities, such as with Bubble Sort (Figure 13) and Radix Sort (Figure 19), is likely due to the details in the way our algorithms were implemented, with certain data structures requiring more time to run and certain datasets requiring more computations to sort.



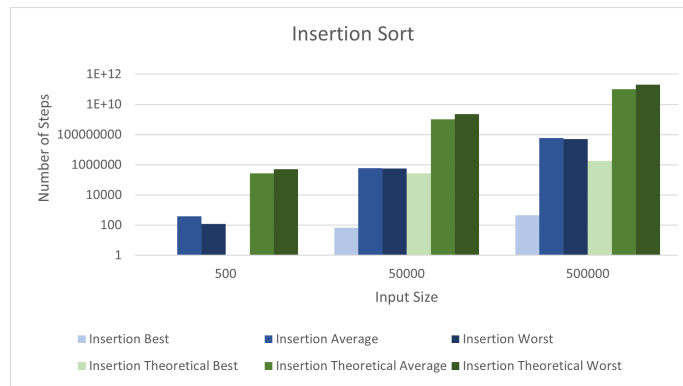


Fig. 4: Insertion Sort: Expected Steps vs. Results

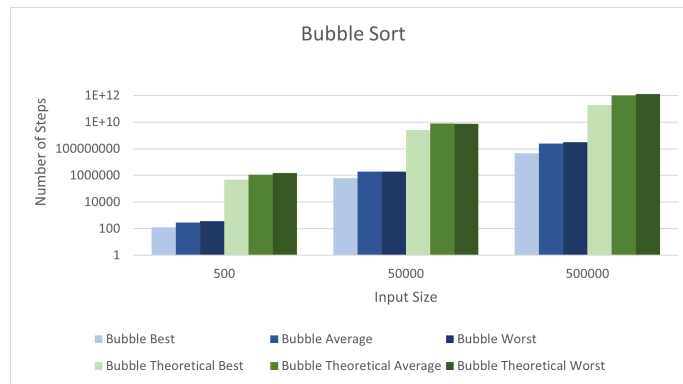


Fig. 5: BubbleSort: Expected Steps vs. Results

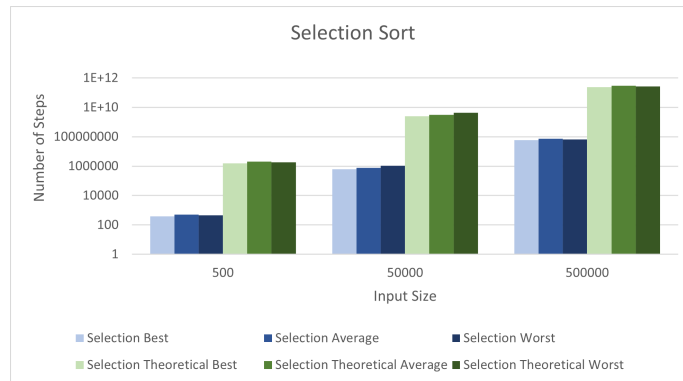


Fig. 6: Selection Sort: Expected Steps vs. Results

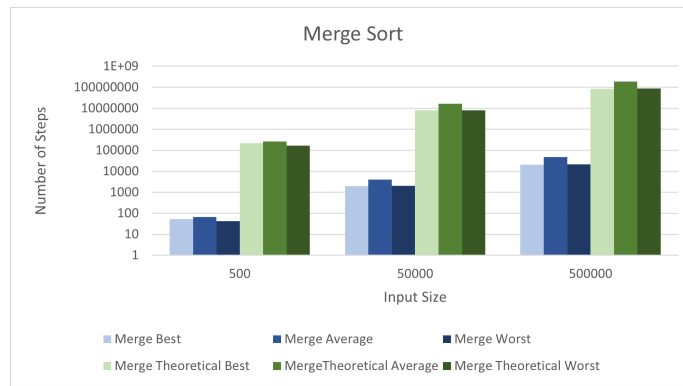


Fig. 7: MergeSort: Expected Steps vs. Results

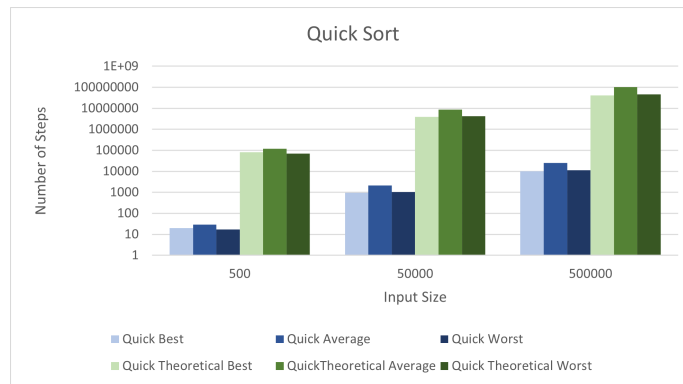


Fig. 8: QuickSort: Expected Steps vs. Results

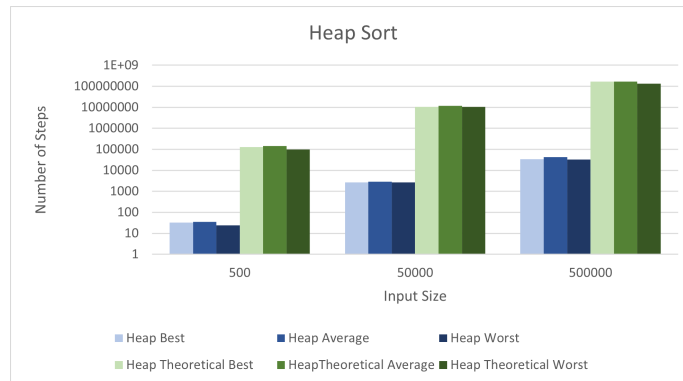


Fig. 9: HeapSort: Expected Steps vs. Results

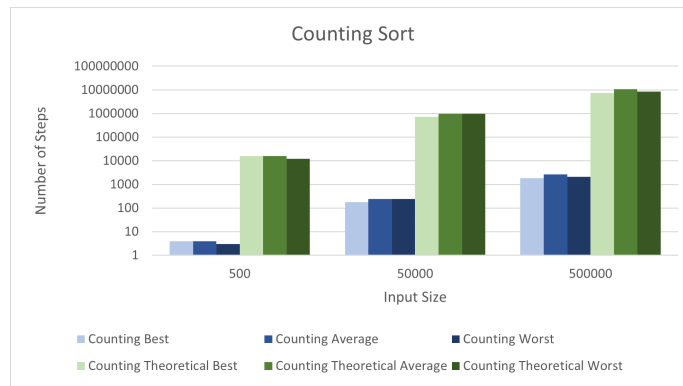


Fig. 10: Counting Sort: Expected Steps vs. Results

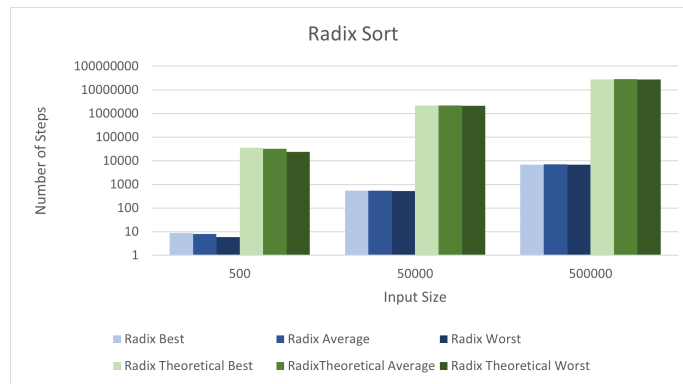


Fig. 11: Radix Sort: Expected Steps vs. Results

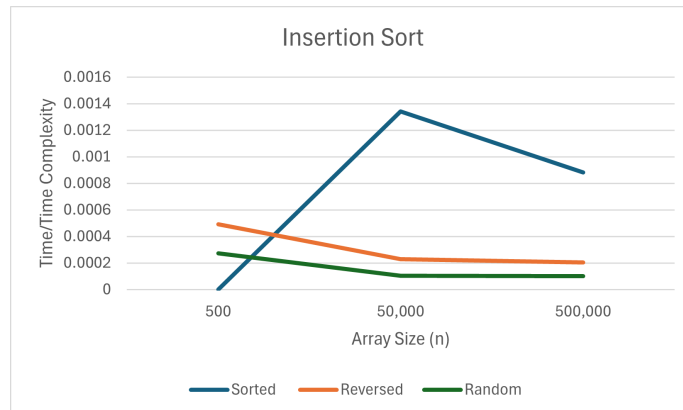


Fig. 12: Insertion Sort: Test of Asymptote

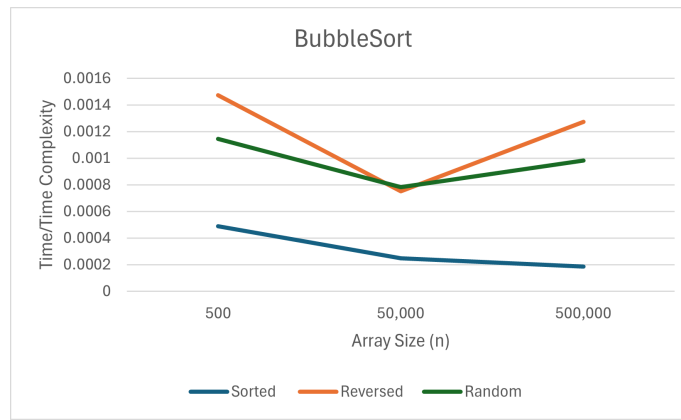


Fig. 13: BubbleSort: Test of Asymptote

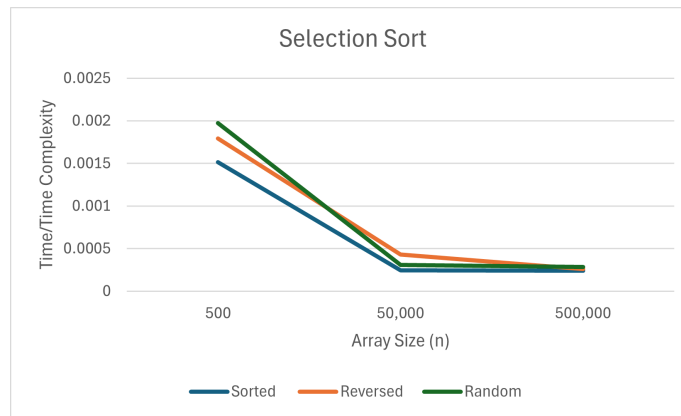


Fig. 14: Selection Sort: Test of Asymptote

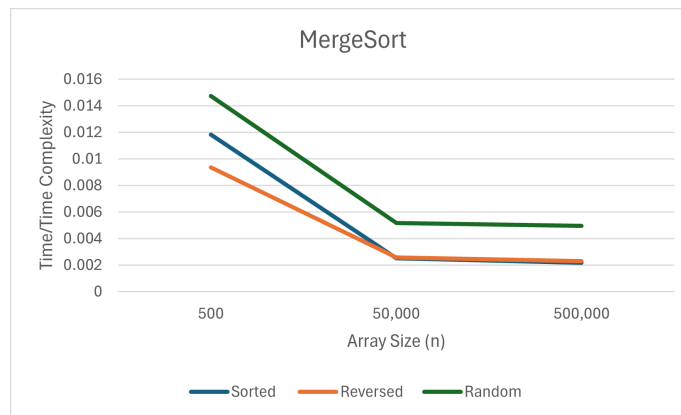


Fig. 15: MergeSort: Test of Asymptote

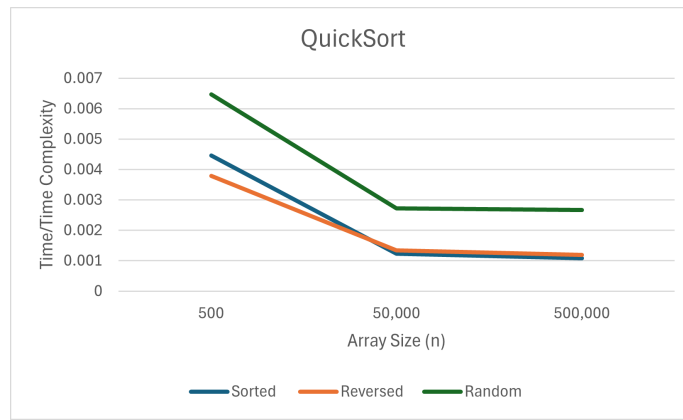


Fig. 16: QuickSort: Test of Asymptote

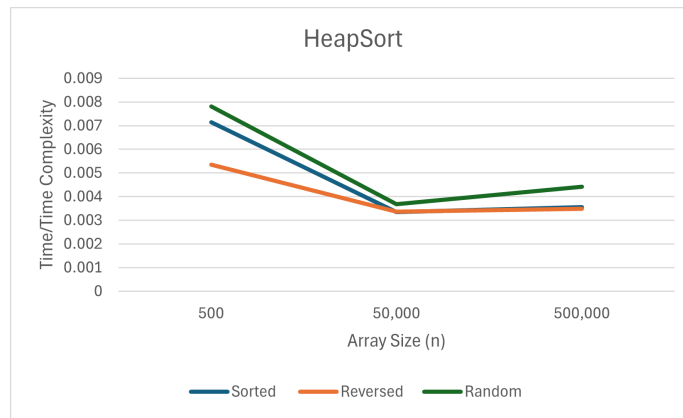


Fig. 17: HeapSort: Test of Asymptote



Fig. 18: Counting Sort: Test of Asymptote

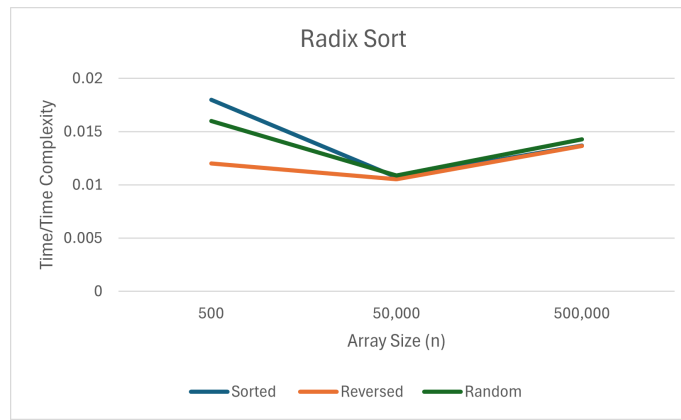


Fig. 19: Radix Sort: Test of Asymptote

### C. Number of Comparisons and Execution Time

Although the number of comparisons gives a broad estimate of the efficiency of algorithms, allowing algorithms to be compared with each other, the number of comparisons is not a good predictor of runtime as many other factors determine real-life algorithm efficiency.

## V. CONCLUSION

In this paper, we discussed the theoretical time complexities of eight distinct sorting algorithms. Next, we designed an experiment to determine the actual time it takes for the sorting algorithms to sort a variety of lists in order to compare the experimental results to the ones expected using time complexities. We found that algorithms with similar theoretical time complexities often have subtle yet pervasive differences in speed in real-life situations due to both the specifics of their implementation and the properties of the dataset they are applied on. Insertion Sort performed the best on pre-sorted arrays whereas Counting Sort performed best in other cases. In our experiments, the algorithms generally followed the theoretical time complexities closely and the theoretical time complexities generally acted as an upper bound for the experimental runtimes. However, we wouldn't go so far as to say that the number of comparisons is a good choice for predicting execution time as the number of comparisons only gives a rough estimate of algorithm efficiency and many other factors determine runtime such as the data structures used in implementation, the properties of the dataset being sorted, and the nature of the device running the algorithms.

## REFERENCES

- [1] L. C. R. R. . S. C. Cormen, T., "introduction to algorithms", "Mit Press", vol. "3", "2019".

## Part 2

### INTRODUCTION

In order to utilize our new knowledge of creating time efficient algorithms, we designed and implemented two algorithms that, given a set  $S$  of  $n$  integers and another integer  $x$ , determines whether or not there exist two elements in  $S$  whose sum is exactly  $x$ .

#### A. Brute Force Approach

For every pair of numbers in the set, we check if their sum equals the target  $x$ . We do this by trying all possible combinations of pairs using a doubly nested for loop. Due to the doubly nested for loops, this approach has a time complexity of  $O(n^2)$ .

| Pseudocode                         | Line |
|------------------------------------|------|
| function hasPairWithSum( $S, x$ ): | 1    |
| $n = \text{length}(S)$             | 2    |
| for $i$ from 0 to $n - 1$          | 3    |
| for $j$ from $i + 1$ to $n - 1$ :  | 4    |
| if $S[i] + S[j] == x$ :            | 5    |
| return true                        | 6    |
| return false                       | 7    |

Algorithm 1: Brute Force Approach

#### B. Efficient Algorithm

Instead of trying all pairs of numbers, we ‘remember’ the numbers we’ve seen so far, and check if the difference between the target and the current number exists in our memory. First, we iterate through the set once, which takes  $O(n)$  time. For each element, we perform a constant time operation of checking if the complementary value exists in the set, which also takes  $O(1)$  time on average in a hash set. Thus, overall, this method takes time proportional to  $O(n)$ , making it much faster than the brute-force method.

| Pseudocode                             | Line |
|----------------------------------------|------|
| function hasPairWithSum2( $S, x$ )     | 1    |
| $\text{seen} = \text{set}()$           | 2    |
| for num in $S$ :                       | 3    |
| if $x - \text{num}$ in $\text{seen}$ : | 4    |
| return true                            | 5    |
| $\text{seen.add}(\text{num})$          | 6    |
| return false                           | 7    |

Algorithm 2: Efficient Algorithm

### CONCLUSION

In the first portion of the report, we noticed that methods that require doubly nested for loops in order to compare all combinations of elements, such as Bubble Sort, tend to have longer runtimes. This is exemplified in our brute force approach to the problem of finding numbers that sum to a given number. We produced a theoretically more efficient algorithm by reducing the number of elements our target sum is compared to.